

KUSHO AI – FULL SPEAKER SCRIPT

Slide 1 – Title

Good morning everyone.

Thank you for joining this session.

Today I will be presenting my evaluation of Kusho AI, an AI-powered API testing platform. The purpose of this evaluation was to understand how Kusho AI can help improve our testing process, where it provides value, and where traditional automation frameworks are still required.

During this presentation, I will cover:

- An overview of Kusho AI
- Areas where it performs well
- Limitations identified during evaluation
- Comparison with traditional automation approaches
- Recommendations and next steps

Let's begin.

Slide 2 – Executive Summary

If I summarize the entire evaluation in one sentence:

"Kusho AI is a strong accelerator for API testing, but it is not a replacement for automation engineers."

The platform performs particularly well in:

- API test generation
- Coverage expansion
- Regression suite creation
- API workflow testing

However, it still has limitations around:

- Business-rule validation
- Enterprise data-driven testing
- Backend validation
- Complex business workflows

Therefore, I see Kusho AI as a productivity enhancement tool rather than a complete testing solution.

Slide 3 – Agenda

Today's discussion is divided into five sections:

First, I will provide an overview of Kusho AI.

Next, I will discuss areas where Kusho AI excels.

Then, I will cover the limitations observed during evaluation.

After that, I will discuss ROI and practical considerations.

Finally, I will share recommendations and next steps.

Slide 4 – Current QA Challenges

Before discussing Kusho AI, let's first understand the challenges we face in traditional API testing.

Typically, when a new API is delivered, the process involves:

- Understanding requirements
- Reviewing Swagger documentation
- Creating test scenarios
- Writing automation scripts
- Executing tests
- Maintaining scripts over time

This process can be time-consuming.

Additionally, testers often focus on common scenarios due to project timelines.

For example, consider a Money Transfer API.

Most testers would validate:

- Successful transfer
- Invalid account number
- Insufficient balance

However, scenarios such as:

- Null values
- Negative values
- Extremely large amounts
- Invalid data types

may be missed.

This is where AI-based testing tools attempt to provide value.

Slide 5 – What is Kusho AI?

Kusho AI is an AI-powered API testing platform.

Instead of manually designing test cases, users provide:

- Swagger/OpenAPI specifications
- Postman Collections
- API endpoints

The AI analyzes the API structure and automatically generates test scenarios.

The primary goal is to reduce manual effort while increasing test coverage.

This can significantly accelerate testing activities, especially during API onboarding and regression testing.

Slide 6 – How Kusho AI Works

The workflow is fairly straightforward.

First, we provide the API specification.

Kusho AI analyzes:

- Request payloads
- Response payloads
- Mandatory fields
- Data types
- Validation constraints

Based on this analysis, it generates test cases and executes them.

Finally, it provides reports and execution results.

In simple terms:

API Specification → AI Analysis → Test Generation → Execution → Reporting

Slide 7 – Key Features

Some of the key features observed during evaluation include:

- Automatic test generation
- AI-generated edge cases
- Regression suite creation
- API chaining
- Automated execution
- Test reporting

These features reduce the amount of repetitive manual work performed by testers.

Slide 8 – Rapid Test Generation

This is one of the strongest capabilities of Kusho AI.

Traditionally, creating test scenarios may take several hours or even days depending on API complexity.

With Kusho AI, test generation can happen within minutes.

For example:

Suppose we have a Customer Registration API.

A manual tester may create 10 to 15 scenarios.

Kusho AI can automatically generate:

- Valid requests
- Missing field validations
- Invalid data types
- Boundary conditions
- Unexpected inputs

This significantly reduces test design effort.

Slide 9 – Coverage Expansion

Another area where Kusho AI performs well is coverage expansion.

Human testers naturally focus on expected business flows.

AI tends to explore more combinations.

For example:

For an Amount field:

Manual testing may use:

- 100
- 500
- 1000

Kusho AI may additionally test:

- 0
- -1
- NULL
- Decimal values
- Maximum values
- Invalid formats

This often results in broader API coverage.

Slide 10 – API Chaining

Kusho AI also supports API chaining.

This means data generated from one API can be used by another API.

For example:

Create Customer

↓

Create Account

↓

Deposit Funds

↓

Verify Balance

This helps simulate realistic workflows and reduces the effort required to manually connect APIs.

However, as we will discuss later, API chaining is different from validating business workflows.

Slide 11 – Productivity Benefits

One of the biggest benefits is productivity improvement.

Instead of spending significant time writing boilerplate test scripts, QA engineers can focus on:

- Business validation
- Risk analysis
- Exploratory testing
- Test strategy

The AI handles much of the repetitive scenario generation activity.

Slide 12 – Kusho AI vs Rest Assured

This is an important comparison because many teams already use Rest Assured.

Rest Assured provides:

- Full flexibility
- Complete customization
- Database integration
- Data-driven testing
- Enterprise framework capabilities

Kusho AI provides:

- Faster test generation
- Reduced maintenance effort
- AI-generated coverage

My view is that these tools serve different purposes.

Rest Assured is a framework.

Kusho AI is an accelerator.

They can work together rather than compete with each other.

Slide 13 – Business Validation Limitation

Now let's discuss limitations.

One major limitation is business-rule understanding.

Kusho AI understands APIs very well.

However, it does not automatically understand business intent.

For example:

Suppose a bank has a rule:

"If customer age is above 60 and account type is Premium, apply a 15% discount."

The API may return a successful response.

But determining whether the discount logic is correct requires business knowledge.

This still requires QA expertise.

Slide 14 – Data-Driven Testing Limitation

Another limitation is enterprise-level data-driven testing.

Our traditional frameworks can:

- Read Excel files
- Read CSV files
- Query databases
- Execute thousands of records
- Write results back

For example:

10,000 rows in Excel

↓

Execute API requests

↓

Store results



Generate reports

This type of highly customized testing is not Kusho AI's primary strength.

Slide 15 – Backend Validation Limitation

In enterprise applications, API validation alone is often insufficient.

We may also need to verify:

- Database updates
- Kafka messages
- MQ messages
- Generated files
- Downstream systems

For example:

An API may return Success.

However, we still need to verify whether the transaction was actually stored in the database.

Such validations often require additional automation frameworks.

Slide 16 – Environment Dependency

Kusho AI performs best when:

- APIs are available
- Swagger specifications exist
- Postman collections are available

If we only have:

- Requirement documents
- Sample JSON files
- Static response samples

the platform's capabilities become more limited.

Therefore, environment readiness is important for successful adoption.

Slide 17 – Evaluation Challenges

During evaluation, a few challenges were encountered.

First, the free version limited access to advanced features.

As a result, pipeline integration capabilities could not be fully evaluated.

Second, a dedicated development environment was unavailable.

Third, some extension-related execution issues prevented complete validation of all scenarios.

These challenges should be considered when interpreting the evaluation results.

Slide 18 – ROI Analysis

Now let's discuss ROI.

Consider a project containing 100 APIs.

Traditionally:

- Test design may require several weeks
- Automation creation may require several more weeks
- Maintenance effort continues throughout the project lifecycle

Kusho AI can significantly reduce the initial effort required for scenario generation.

Even a 50% reduction in test creation effort can generate meaningful productivity improvements.

This is where the business value of the platform becomes visible.

Slide 19 – Recommendation

Based on this evaluation, my recommendation is to adopt Kusho AI selectively.

Recommended use cases:

- New API onboarding
- Regression suite generation

- Coverage enhancement
- API workflow validation

Continue using traditional frameworks for:

- Business-rule validation
 - Data-driven testing
 - Database verification
 - Enterprise integrations
 - Complex automation requirements
-

Slide 20 – Conclusion

To conclude,

Kusho AI is a promising platform that can significantly accelerate API testing activities.

Its biggest strengths are:

- Speed
- Coverage
- Automation of repetitive testing tasks

Its biggest limitations are:

- Business-rule understanding
- Enterprise-level customization
- Backend validation

Therefore, I recommend viewing Kusho AI as a complementary tool that works alongside traditional automation frameworks rather than replacing them.

Thank you for your time.

I am happy to answer any questions.

Expected Questions

Q: Can Kusho AI replace Rest Assured?

Answer: No. Rest Assured provides much greater flexibility and customization. Kusho AI should be viewed as a productivity accelerator rather than a replacement.

Q: Can Kusho AI perform API chaining?

Answer: Yes. Kusho AI supports API chaining and multi-step API workflows.

Q: Can Kusho AI perform large-scale Excel-driven testing?

Answer: It is not designed primarily for enterprise-level Excel-driven frameworks involving thousands of records and custom reporting.

Q: Should we adopt Kusho AI?

Answer: Yes, for API onboarding, coverage expansion, and regression acceleration. Traditional automation frameworks should still be maintained for complex business scenarios.